

Dev Team — Manual de Usuario (v2.2)

Equipo completo de desarrollo con **15 agentes IA** que cubre todo el ciclo de vida del software: Historia de Usuario → diseño UI/UX → arquitectura → desarrollo → QA en equipo con evidencia → seguridad → **pases a ambientes** → deploy → documentacion, con una **wiki viva** del proyecto y una **oficina virtual en vivo** para ver al equipo trabajar. Funciona con proyectos **nuevos o existentes**, en **mono-repo o multi-repo**, con backlog en **GitHub o Azure DevOps**.

Indice

1. [Instalacion](#)
 2. [Inicio rapido](#)
 3. [El equipo: 15 agentes](#)
 4. [Comandos \(22\)](#)
 5. [Flujos de trabajo](#)
 6. [La wiki del proyecto](#)
 7. [La oficina virtual](#)
 8. [Metricas del equipo](#)
 9. [Pases a ambientes](#)
 10. [Configuracion](#)
 11. [Preguntas frecuentes](#)
-

Instalacion

```
# 1. Agregar el marketplace (una sola vez)
/plugin marketplace add faast-app/faast-claude-marketplace

# 2. Instalar el plugin
/plugin install dev-team@faast-marketplace

# 3. Actualizar cuando haya cambios
/plugin marketplace update faast-marketplace
/plugin update dev-team
```

Inicio rapido — un solo comando

No necesitas memorizar nada. Escribe:

```
/dev-team:start
```

El equipo detecta tu situacion y te guia:

- **No hay proyecto** → te ofrece crear uno nuevo o heredar uno existente
- **Hay proyecto** → te muestra el estado y te propone la siguiente accion
- **Pediste algo especifico** → `/dev-team:start` quiero agregar un filtro de fechas y el equipo enruta tu pedido al agente correcto

Antes de cualquier trabajo, el agente **setup** valida que tengas todo instalado (git, Docker, gh/az, cliente de BD, Playwright, Node) y **ofrece instalar lo que falte**. Nunca te vas a topar con un error criptico por una herramienta faltante.

El equipo: 15 agentes

Agente	Rol	Modelo
 setup	Valida e instala prerequisites, configura conexiones BD y tracker	haiku
 product-owner	HUs de negocio con criterios de aceptacion; items de entregable ricos en GitHub/Azure	sonnet
 architect	Arquitectura: topologia (mono/multi), servicios, stack, BD, gateway	opus
 ui-designer	Mockups y specs UI/UX: paletas (contraste WCAG), tipografia, estados, accesibilidad	sonnet
 lead	Coordina, asigna, exige gates de calidad, unico que mergea	sonnet
 backend	Servicios backend (.NET 8, Node.js, Python, Java)	sonnet
 frontend	SPA y microfrontends (React, Vue, Angular)	sonnet
 dba	Esquemas, indices, migraciones, scripts de pase, comparacion de BDs	sonnet
 qa (QA Lead)	Plan de pruebas, reparte a los especialistas, consolida el veredicto, suite E2E	sonnet
 qa-frontend	Especialista QA de UI: Playwright MCP, responsive, evidencia visual	sonnet
 qa-backend	Especialista QA de APIs: contract testing contra OpenAPI, permisos, bordes	sonnet
 release-manager	Solicitudes de pase (Word+PDF), audita los scripts del DBA , Scripts.zip	sonnet
 infra	Docker, CI/CD, gateways, deploy	sonnet
 cybersec	Audita seguridad; nunca commitea, reporta hallazgos	sonnet
 tech-writer	Docs tecnicas + mantenedor de la wiki del proyecto	haiku

Los modelos estan asignados para optimizar costo: **opus** solo donde el razonamiento complejo lo amerita, **haiku** para trabajo mecanico, **sonnet** para el resto. Nunca se usa fable.

Reglas del equipo (siempre activas)

- **El PO redacta TODO en lenguaje funcional de negocio** — HUs, bugs e items que entiende QA o cualquier persona no programadora: sin código, sin jerga, títulos limpios sin códigos raros (parámetros funcionales sí, como los ve el usuario). Lo técnico va al final o en los handoffs, nunca en la narrativa
- **Se trabaja con Scrum** — sprints con Sprint Goal, story points (1-8), backlog priorizado por valor, refinamiento antes del planning, review y retrospectiva
- **QA es un equipo y es gate de merge** — nada llega a main sin veredicto APROBADA
- **QA no debuggea** — reproduce, documenta y reporta (bloqueantes de inmediato), SIEMPRE con evidencia: screenshots o clips cortos, adjuntos al ítem del tracker
- **Cybersec es segundo gate** en features que tocan auth o datos sensibles y nunca commitea
- **Solo el Lead mergea** a develop/main; un agente = un branch = una tarea
- **El release-manager es gate de pases** — audita el formato de los scripts del DBA y puede rechazarlos hasta que cumplan
- **Los devs preguntan antes de abrir PR** — no todo entregable lleva PR
- **Solo el Lead delega en subagentes** — puede lanzar a cualquier agente del equipo, incluso varias instancias del mismo rol en paralelo (2 backend en HUs distintas), nunca otro lead. Los demás agentes ejecutan su trabajo directamente (herramienta deshabilitada + hook que bloquea el intento; única excepción: Explore, búsqueda barata de solo-lectura). Esto evita el desperdicio de tokens de un backend lanzando 3 backends anidados
- **Wiki primero** — todos los agentes leen `.coordination/wiki/` antes de cada tarea (menos tokens, más contexto); solo el tech-writer la escribe
- **Todo queda registrado AUTOMATICAMENTE** — hooks del plugin escriben `task_start/task_end` en `.coordination/metrics/activity.jsonl` cada vez que un agente arranca o termina (sin depender de que el agente se acuerde); los agentes agregan lo demás (handoffs, bloqueos, evidencia). Alimenta métricas y la oficina virtual

Comandos

Los 3 que usaras todos los días

Comando	Que hace
<code>/dev-team:start</code>	Punto de entrada universal: detecta contexto y te guía
<code>/dev-team:status</code>	Estado del proyecto: sprint, backlog, handoffs, bloqueos
<code>/dev-team:sync</code>	Sincroniza con GitHub/Azure: trae tickets, sube avances, crea PRs

Crear o adoptar proyectos

Comando	Que hace
<code>/dev-team:new-project {doc o idea}</code>	Proyecto nuevo: arquitectura → repos → backlog → wiki
<code>/dev-team:onboard {nombre}</code>	Hereda un proyecto existente (detecta repos, trae tickets)
<code>/dev-team:setup</code>	Valida/instala prerequisites (<code>setup db</code> , <code>setup tracker</code> , <code>setup playwright</code>)

Backlog y trabajo

Comando	Que hace
<code>/dev-team:refine {pedido}</code>	El PO convierte tu pedido en HUs y las crea en el tracker
<code>/dev-team:assign-task</code>	El Lead asigna HUs/tickets a los agentes
<code>/dev-team:inbox</code>	Un agente lee sus tareas y handoffs pendientes
<code>/dev-team:handoff</code>	Crear comunicacion entre agentes

Calidad

Comando	Que hace
<code>/dev-team:test-plan {HU}</code>	QA genera el plan de pruebas desde los criterios
<code>/dev-team:e2e {HU run explorar url}</code>	QA valida con Playwright y automatiza la suite E2E
<code>/dev-team:review-pr {n}</code>	Revision de codigo de un PR
<code>/dev-team:security-audit</code>	Auditoria de seguridad
<code>/dev-team:git-check</code>	Verificacion git antes de commitear

Operacion y releases

Comando	Que hace
<code>/dev-team:db-health</code>	Health check de BD (esquema, indices, slow queries)
<code>/dev-team:deploy-check</code>	Verifica readiness para deploy
<code>/dev-team:pase {ambiente}</code>	Solicitud de pase completa: documento Word+PDF + Scripts.zip auditado
<code>/dev-team:document {tema}</code>	Tech Writer actualiza documentacion tecnica

Conocimiento y visibilidad

Comando	Que hace
<code>/dev-team:wiki {init ingest lint query}</code>	Wiki viva del proyecto (vault de Obsidian)
<code>/dev-team:team-office</code>	Oficina virtual 2D en vivo — ve al equipo trabajar
<code>/dev-team:team-metrics</code>	Productividad y consumo de tokens por agente

Flujo A: Proyecto nuevo

```
/dev-team:new-project docs/requerimientos.docx
```

1. **setup** valida tu entorno (instala lo que falte, con tu OK)
2. **architect** analiza los requerimientos y propone: topologia (mono o multi-repo), servicios, stack, BD, gateway — con diagrama y justificacion
3. **Tu apruebas o ajustas** ("prefiero mono-repo", "usa MySQL", "quita el servicio X")
4. Eliges tracker: **GitHub Projects** o **Azure DevOps Boards**
5. Se crean los repos/carpetas con scaffolding completo (Dockerfile, CI/CD, tests)
6. **product-owner** convierte los requerimientos en HUs reales en tu tracker
7. Se crea la **wiki** (/wiki init) y la carpeta de metricas
8. Listo: **/dev-team:assign-task** para empezar a trabajar

Flujo B: Heredar un proyecto existente

```
/dev-team:onboard backoffice
```

1. **setup** valida entorno y pregunta: ¿GitHub, Azure DevOps o solo local?
2. Detecta los repos (locales y remotos), analiza stack y estructura de cada uno
3. Detecta la topologia (mono/multi) — **nunca propone migrarla**
4. Configura el acceso del DBA a la BD (motor, credenciales, prueba de conexion)
5. Trae los tickets reales del tracker y arma el backlog
6. Crea la wiki e ingiere lo detectado (arquitectura, backlog, mapa de repos)
7. Listo: el equipo conoce tu proyecto y puede trabajar

Flujo C: Una feature de principio a fin

```
/dev-team:refine "los analistas necesitan filtrar cobranzas por fecha"
```

1. PO escribe la HU (negocio + criterios Gherkin) → la crea en GitHub/Azure
2. (Si hay pantallas nuevas) ui-designer propone mockups → tu eliges → spec a frontend
3. Lead asigna: backend + frontend implementan → QA prepara el plan de pruebas en paralelo
 - Los devs preguntan si el entregable lleva PR antes de abrir nada
 - Rama SIEMPRE desde origin/{defaultBranch} + version bump + UN PR consolidado
4. Devs terminan → handoff a QA
5. QA Lead reparte: qa-frontend valida criterios de UI y qa-backend los de API — EN PARALELO
 - Todo con evidencia (screenshots/clips) · sin debuggear · bloqueantes al instante
6. Cybersec audita (solo si toca auth/datos sensibles)

7. Lead verifica gates: CI verde + QA aprobo + seguridad OK → merge
8. /dev-team:sync push → PR mergeado, HU a Done, descripcion rica item→PR alineada
9. tech-writer documenta e ingiere todo a la wiki

Flujo D: Reportar y corregir un bug

```
/dev-team:start hay un bug en la paginacion de cobranzas
```

1. **Lead** hace triaje: ¿que componente falla? ¿severidad?
2. **QA** lo reproduce (sin debuggear): pasos exactos + screenshots/clip
3. El PO registra el Bug en el tracker **con la evidencia adjunta**; el Lead lo asigna con branch **fix/...**
4. El dev corrige (la causa raiz es SU trabajo, no de QA); **QA escribe el test de regresion** que cubre el bug
5. Lead mergea cuando el test pasa; el bug no vuelve

Flujo E: Pruebas

```
/dev-team:test-plan HU-042      # plan de pruebas desde los criterios
/dev-team:e2e HU-042           # valida y automatiza esa HU
/dev-team:e2e run               # corre la regresion completa
/dev-team:e2e explorar http://localhost:3000 # prueba exploratoria libre
```

El equipo QA trabaja en dos modos:

- **Interactivo** (Playwright MCP): navega la app real, valida criterios, captura evidencia
- **Automatizado** (suite Playwright): tests en codigo que corren en CI en cada PR

Cada criterio de aceptacion = al menos un test (trazabilidad HU → criterio → test). La evidencia queda en **.coordination/evidence/{HU|BUG}/** y, si es un bug, adjunta al item del tracker.

Flujo F: Base de datos

```
/dev-team:db-health           # configura conexion si falta, luego health
check
/dev-team:db-health full      # esquema, indices, slow queries, tamaños
```

Ademas el DBA puede:

- **Comparar dos bases de datos** (dev vs cert, cliente A vs cliente B): schema diff, charset, data de catalogos y volúmenes — en modo **estrictamente solo-lectura** (jamás escribe en las BDs comparadas). Pideselo via **/dev-team:start compara la BD de dev contra la de certificacion**

- **Preparar scripts de pase** en el formato global: archivos numerados por tipo (`1_createTable.sql ... 7_update.sql`), 100% idempotentes (re-ejecutables N veces), insert-only, DB-agnosticos, con acentos/ñ intactos (UTF-8 verificado)

Flujo G: Pase a un ambiente

```
/dev-team:pase preprod PE
```

1. El **release-manager** recopila: ambiente destino (si es productivo, ¿que cliente?), componentes y versiones, ¿lleva scripts de BD?
2. Si lleva scripts: **audita el paquete del DBA** contra el checklist del formato global — si algo falla, lo **rechaza y devuelve** al DBA con archivo+linea+regla
3. Con auditoria aprobada: consolida **Scripts.zip** (los .sql numerados en la raiz)
4. Llena la **solicitud de pase** desde la plantilla oficial y la exporta a PDF
5. Entrega la **carpeta de pase** completa:

```
Release v1.0.0 09julio2026 - Notificaciones Cobranza/  
├─ Solicitud de Pase Ambientes - Preprod PE.pdf  
├─ Solicitud de Pase Ambientes - Preprod PE.docx  
└─ Scripts.zip
```

El documento es **obligatorio** para: certificacion, puente, demo (Chile/Peru/ Colombia), preprod (CO/PE) y productivos de cliente. Para ambientes internos, solo si lo pides.

Flujo H: Diseño de pantallas

```
/dev-team:start necesito la pantalla de resumen de cobranzas
```

1. **ui-designer** detecta tu sistema de diseño actual (tailwind config, tokens, pantallas existentes via screenshot)
2. Te entrega **2-3 propuestas de mockup** (HTML que abres en el browser) + spec completa: paleta con contraste WCAG verificado, tipografia, espaciado, estados (loading/error/empty/success), responsive
3. Eliges una → la spec pasa a **frontend** para implementar exactamente eso

La wiki del proyecto (patron LLM Wiki)

`.coordination/wiki/` es la **memoria viva del proyecto** (patron LLM Wiki de Karpathy): en vez de que cada agente re-lea decenas de handoffs viejos, el **tech-writer destila** todo a paginas canonicas enlazadas con `[[wikilinks]]`.

```
wiki/
├── index.md           # portada con el mapa del proyecto
├── servicios/         # estado vivo de cada servicio (stack, contratos,
BD)
├── hus/              # una pagina por HU con su historia completa
├── bugs/             # reproduccion, evidencia, fix
├── decisiones/       # ADRs
├── pases/            # que se paso, a donde, con que scripts
└── agentes/         # memoria por rol
```

Comandos:

```
/dev-team:wiki init      # crear la wiki (una vez; new-project/onboard lo
hacen solos)
/dev-team:wiki ingest    # destilar los handoffs/reportes nuevos (hazlo al
final del dia)
/dev-team:wiki lint      # salud: links rotos, huerfanos, paginas
desactualizadas
/dev-team:wiki query ¿por que elegimos MySQL para cobranzas? # responde
citando paginas
```

Con Obsidian: abre [.coordination/wiki/](#) como vault → tienes **graph view** del conocimiento del proyecto (HUs ↔ servicios ↔ bugs ↔ decisiones) gratis.

Por que importa: cada **ingest** (lo hace tech-writer, que es haiku = barato) hace que la siguiente tarea de cualquier agente sea mas barata y con mejor contexto. El conocimiento compone.

La oficina virtual (team-office)

```
/dev-team:team-office
```

Abre <http://localhost:4321>: una **oficina 2D en vivo** (estilo Gather Town) con los 15 agentes en sus salas:

-  **Trabajando:** arco giratorio, burbuja de "escribiendo...", monitor parpadeando y su tarea actual debajo
-  **Bloqueado:** anillo rojo pulsante con 
-  **Handoffs:** el agente **camina** hasta el escritorio del destinatario a entregar el sobre ( pop al recibir)
-  **Tarea completada:** confetti
- Panel lateral: equipo ordenado por estado, handoffs pendientes, feed de actividad
- Header: contadores en vivo (trabajando / bloqueados / handoffs / eventos)

Controles: rueda = zoom · arrastrar = mover · doble-click = vista general · click en un agente = la camara lo sigue.

Como funciona: un servidor Node local sin dependencias que solo LEE `.coordination/` (eventos de `metrics/activity.jsonl` + carpeta `handoffs/`) y empuja cambios por SSE. Cero tokens, cero escritura.

El registro es automatico: los hooks del plugin (`SubagentStart/SubagentStop`) escriben el evento cada vez que un agente dev-team arranca o termina — veras a los agentes encenderse en la oficina sin que ellos hagan nada. Requiere reiniciar la sesion de Claude Code tras instalar/actualizar el plugin (los hooks se cargan al inicio).

Metricas del equipo (team-metrics)

```
/dev-team:team-metrics          # dashboard del sprint
/dev-team:team-metrics --watch   # modo live
```

El Lead genera un ranking por agente: tareas completadas, handoffs, commits, **lead time real** (task_start → task_end del event log), tokens consumidos y costo estimado segun el modelo de cada agente. Cierra con recomendaciones (rebalancear carga, bajar/subir modelo de un agente, handoffs estancados).

Fuente primaria: `metrics/activity.jsonl`. Los tokens se estiman desde los transcripts de Claude Code; si no estan disponibles se reporta "sin datos" — nunca cifras inventadas.

Pases a ambientes (pase)

Resumen de reglas que el equipo aplica solo (detalle en el Flujo G):

Regla	Detalle
Scripts agrupados por TIPO	1_createTable 2_alterTable_add 3_alterTable_modify 4_views 5_insertInto 6_procedures 7_update
Idempotencia total	Re-ejecutable N veces: IF NOT EXISTS, guards por information_schema, INSERT solo con WHERE NOT EXISTS
Insert-only	Prohibido ON DUPLICATE KEY UPDATE / REPLACE INTO — un pase jamas toca datos ya cargados
DB-agnostico	Sin mi_db.tabla; corre contra la BD conectada de cualquier ambiente/cliente
FKs externos por natural key	Nunca ids literales de catalogos (roles, monedas...) — cambian por ambiente
Acentos intactos	UTF-8 en todo; deteccion de mojibake antes de entregar

Regla	Detalle
Gate de auditoria	El release-manager verifica TODO el checklist; si falla, rechaza y el DBA corrige
Entregable	Carpeta Release vX.Y.Z {fecha} – {Proyecto} / con PDF + Word + Scripts.zip

Configuracion del proyecto

Todo vive en **.coordination/config.json** (lo crean new-project/onboard):

```
{
  "project": "backoffice",
  "topology": "multi",
  "tracker": {
    "provider": "azure",
    "azure": { "org": "faast", "project": "BackOffice" },
    "reviewer": "carlos.fuentes@faast.app",
    "overheadEpicId": 1234,
    "areaPath": "BackOffice\\Fintec",
    "iterationPath": "BackOffice\\Sprint 12"
  },
  "git": {
    "defaultBranch": "develop",
    "identity": { "name": "Carlos Fuentes", "email":
"carlos.fuentes@faast.app" }
  },
  "pase": {
    "templatePath": "(opcional – default: plantilla del plugin)",
    "outputDir": "(opcional – default: .coordination/pases/)"
  },
  "urls": { "dev": "http://localhost:3000" }
}
```

Campo	Efecto
topology	mono/multi : como trabajan los agentes y donde vive .coordination/
tracker.provider	github/azure : donde viven HUs, items y PRs
tracker.reviewer	Reviewer que los devs ponen en cada PR
tracker.overheadEpicId	Epica/PBI padre para fixes sueltos sin epica propia
git.defaultBranch	Rama base OBLIGATORIA para ramificar (via git fetch origin)
git.identity	Identidad de commits del proyecto (no la default del agente)
pase.*	Plantilla y carpeta de salida de las solicitudes de pase

Estructura completa de **.coordination/**:

```
.coordination/
├── config.json           # fuente de verdad (arriba)
├── handoffs/ (+archive/) # comunicacion entre agentes
├── wiki/                # wiki viva (vault de Obsidian)
├── metrics/             # activity.jsonl (eventos de los agentes)
├── evidence/           # screenshots/clips de QA
├── pases/              # carpetas de pase entregadas
├── office/             # la oficina virtual (se instala con /team-office)
├── test-plans/         # planes de prueba de QA
├── backlog.md · sprint-actual.md · architecture.md · repos.md
├── dba-access.json      # credenciales BD – NUNCA en git
└── setup-status.json    # estado del entorno – NUNCA en git
```

Preguntas frecuentes

¿Tengo que saber que agente invoca cada cosa? No. Usa **/dev-team:start** y describe lo que necesitas. El equipo enruta solo.

¿Funciona sin GitHub ni Azure DevOps? Si — elige "solo local" en el onboard. El backlog vive en **.coordination/backlog.md**. Puedes conectar un tracker despues con **/dev-team:setup tracker**.

¿Por que QA no arregla los bugs que encuentra? Por diseño: QA reproduce y reporta con evidencia; el dev responsable corrige. Asi la evidencia es objetiva, el fix lo hace quien conoce el codigo, y QA no pierde tiempo debuggeando.

¿Que evidencia deja QA y donde queda? Screenshots de cada paso y clips cortos (<30s) para lo dinamico, en **.coordination/evidence/**. Si es un bug, se adjunta al PBI/WI/Issue del tracker.

¿Que pasa si el DBA entrega scripts mal formateados para un pase? El release-manager los rechaza con el detalle exacto (archivo, linea, regla) y el DBA corrige. Nada se consolida en Scripts.zip hasta que el checklist pase completo.

¿El DBA puede escribir en las BDs cuando compara dos bases? No. La comparacion es solo-lectura por regla dura. Los scripts de nivelacion se GENERAN como archivos que tu decides cuando ejecutar.

¿Siempre se crea un PR al terminar una tarea? No. Los devs preguntan primero — hay entregables que no llevan PR.

¿Como veo lo que esta haciendo el equipo ahora mismo? **/dev-team:team-office** (oficina visual en vivo) o **/dev-team:status** (texto). Para numeros: **/dev-team:team-metrics**.

¿Necesito Obsidian para la wiki? No — la wiki es markdown puro y los agentes la usan igual. Obsidian solo agrega el graph view y una navegacion comoda para humanos.

¿Que pasa si no tengo Playwright/Docker/gh/Node instalado? El agente setup lo detecta y te ofrece instalarlo. Nada falla en silencio.

¿Puedo usar solo una parte del equipo? Si. Cada comando funciona independiente: solo `/db-health` para BD, solo `/refine` para HUs, solo `/pase` para un release, solo `/e2e` para pruebas.

¿El equipo respeta mi proyecto tal como esta? Si. En onboard nunca se propone cambiar topologia, stack ni convenciones existentes salvo que lo pidas.

¿Como cambio de GitHub a Azure DevOps (o al reves)? Edita `tracker.provider` en `.coordination/config.json` y ejecuta `/dev-team:setup tracker` para validar la autenticacion del nuevo proveedor.